The Fellowship of the Query: Learning Retrieval Actions

Anonymous Author(s)

Abstract

Retrieval-augmented question answering requires control decisions about when to search, reformulate, extract evidence, synthesize facts, verify progress, and stop. We study whether trajectory fine-tuning can improve small language models (SLMs) as next-action controllers, including a low-resource setting where the same SLM is used as both controller and final-answer generator. From accepted teacher search traces, we build a seven-way action-prediction task, where the model predicts the next structured teacher action from the current trajectory state, and evaluate LoRA-supervised fine-tuning across SLMs and xSLMs as controllers. On 1,646 held-out action examples, Granite 4.1 3B trained on 13,194 actions reaches macro-F1 0.6536, compared with 0.1736 for zero-shot prompting of the same model and 0.5399 for a TF-IDF logistic-regression baseline. In an end-to-end controller/generator swap evaluation over 149 held-out trajectories, using the fine-tuned model for both roles improves Exact Match from 0.7530 to 0.7946 and token F1 from 0.7783 to 0.8295 compared with using the base model as both controller and generator. The cross-role conditions show that the fine-tuned controller increases evidence-fact recording when the generator is fixed, while controller-only final-answer gains are not statistically clear. Overall, trajectory supervision improves action prediction and evidence-recording behaviour in this evaluated pipeline. The code used in this paper is available in an anonymous GitHub repository. [1]

CCS Concepts

• **Information systems** → **Question answering**; *Retrieval models and ranking*; • **Computing methodologies** → *Machine learning*.

Keywords

retrieval-augmented generation, small language models, search control, parameter-efficient fine-tuning

1 Introduction

Retrieval-augmented generation (RAG) connects language models to external evidence sources for knowledge-intensive tasks [16, 19]. In multi-hop question answering, retrieval is not a single operation. A system must decide when to decompose a question, search, reformulate a query, extract evidence, synthesize intermediate facts, verify progress, and produce a final answer. This creates a search-control problem: final answer quality depends on both generation and the action sequence that constructs the evidence state.

Prompted agent frameworks such as ReAct combine reasoning and tool use [28]. This can work when the model reliably follows an action protocol, but smaller models may repeatedly search, fail to extract grounded evidence, or fail to emit a valid finishing action. In this paper, we use *xSLMs* for models below 1B parameters and *SLMs* for models in the 1B–3B range. Our central question is whether trajectory supervision can improve an SLM as a next-action controller for search.

Prior work has studied interleaved retrieval and reasoning, retrieval self-reflection, query refinement, and search-agent training. IRCoT interleaves retrieval with chain-of-thought reasoning for multi-step QA [26]. Self-RAG trains models to retrieve, generate, and critique retrieved evidence [3]. RQ-RAG focuses on learned query refinement actions such as rewriting, decomposition, and disambiguation [6]. Search-R1 trains models to interact with search engines over multi-turn retrieval trajectories using reinforcement learning [13]. We study whether lightweight LoRA supervision enables xSLMs and SLMs to imitate a structured seven-action search-control protocol, and whether this changes evidence-fact recording in a constrained multi-hop retrieval pipeline. The accepted teacher trajectories lead to correct answers under our filtering protocol, but we do not assume that every action in those trajectories is optimal; another trajectory could reach the same answer with different or fewer actions.

The seven-action protocol separates recurring decisions in iterative information seeking: clarifying or decomposing the queries, issuing and reformulating queries, collecting evidence, combining evidence, checking progress, and completing the task. This motivation is consistent with prior models that describe search as an evolving and staged process rather than a single query-response event [4, 15, 18]. Action-level imitation under this protocol evaluates whether the model chooses the correct next retrieval action.

We study this question using supervised trajectory fine-tuning. In our setting, *action prediction* is the task of predicting the next structured action using as input the current question, trajectory state, and evidence context; the supervised target is the teacher action stored in an accepted trace. We evaluate this task across multiple SLMs and xSLMs, learning formulations, and training-set sizes. We then place the selected model in an end-to-end BM25 search pipeline [23] and evaluate final answers with SQuAD-style Exact Match and token F1 [22].

This work contributes a focused evaluation of search-flow control for RAG-style question answering as supervised action prediction from structured teacher trajectories. It studies trajectory fine-tuning for SLMs and xSLMs as controllers across selected models, training-set sizes, and multiclass versus binary action formulations [1, 2, 11, 12, 17, 20, 21, 29], then evaluates the selected fine-tuned Granite 4.1 3B model in a repeated end-to-end controller/generator

[1] Code: https://anonymous.4open.science/r/agent-action-controller-42C7/.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM26, Rome

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

swap pipeline with paired bootstrap confidence intervals over the same held-out trajectories.

2 Method

2.1 Trajectory-to-Action Supervision

The supervision source is a set of accepted search trajectories generated by a teacher rollout model, Qwen 3-Next 80B-A3B Instruct [24]. Of 5,000 attempted rollouts, 1,490 trajectories passed the filtering pipeline and were used for training and evaluation. The rejected traces include 2,298 rejected by the gold-answer filter, 850 by deduplication, 254 by repeated-search filtering, and 108 by grounding checks. The resulting evaluation population is therefore conditional on teacher success under this filtering protocol; it is not an unfiltered sample of multi-hop questions. The accepted trajectories contain 16,488 action steps. Each step stores the teacher input messages, teacher JSON output, action tool, action parameters, observation, thought text, original query, dataset identifier, sample identifier, gold answer, and final citations when present.

Each trajectory is converted into supervised examples of the form $(q, s_t, e_t) \mapsto a_t$, where q is the question, s_t is the current structured state represented in the teacher prompt, e_t is the evidence context visible in that prompt, and a_t is the next action emitted by the teacher. The action space contains seven tools: `decompose`, `search`, `reformulate`, `extract`, `synthesize`, `verify`, and `finish`. For LoRA SFT, the model is trained to reproduce the teacher's full JSON action output; for classification methods, the label is the `action.tool` field. For an action at step t, the student input is the stored teacher prompt before that action. It contains the original question, prior state, previous tool history, and evidence preview visible to the teacher at that step. TOther fields like the teacher thought text, gold answer, final answer, final citations, dataset identifier, and sample identifier are excluded from the action-prediction input. Gold answers are used for filtering and final-answer evaluation metadata, not as explicit input fields. The target output for generative controllers is JSON, but we only evaluate how accurately the SLM predicts the correct next action, we ignore the validity of the output JSON since we parse the predicted action.

Splitting is performed at both trajectory and originating-query level: no trajectory and no original query contributes action decisions to any other split to avoid any contamination. The split contains 1,192 train trajectories, 149 validation trajectories, and 149 test trajectories, corresponding to 13,194 train actions, 1,648 validation actions, and 1,646 test actions. The action distributions are similar across splits. For example, `search` is 40.1%, 40.7%, and 42.5% in train, validation, and test, while `verify` is 1.2%, 0.8%, and 1.0%. We report macro-F1 as the primary action-prediction metric, with accuracy and unparsed-output counts as secondary metrics. Unparsed cases are samples where robust action parsing cannot recover a predicted action; outputs are counted as parsed even when the JSON is invalid, as long as the action can be recovered.

2.2 Learning Formats

We evaluate zero-shot prompting, TF-IDF logistic regression, LoRA sequence classification, LoRA supervised fine-tuning (LoRA SFT), and one-vs-rest binary variants. LoRA adapters use rank $r = 16$, $\alpha = 32$, dropout 0.05, no bias terms, and attention projection targets corresponding to query, key, value, and output projections where available [10]. Target module names differ by architecture, but the common design is to adapt attention projections while keeping the base model fixed.

The LoRA SFT models are trained for two epochs with a learning rate of $2 \cdot 10^{-4}$, bfloat16, maximum training length 2,048 tokens, batch size 2, and gradient accumulation 4. Sequence-classification LoRA uses the same learning rate, epochs, precision, and maximum length, with batch size 4 and gradient accumulation 2. Both therefore use an effective batch size of 8 examples. We use the Hugging Face Trainer defaults for AdamW optimisation and a linear schedule with no warmup, evaluate and save once per epoch, keep one checkpoint, and load the best checkpoint by validation loss for LoRA SFT and validation macro-F1 for sequence classification. Generative evaluation uses greedy decoding with maximum input length 3,072 and maximum 512 new tokens. Invalid or unparsed generated actions are counted as unparsed rather than assigned to a valid class.

2.3 End-to-End Pipeline

The end-to-end agent has a controller role and a final-answer generator role. The same model can be used for both roles, or different models can be assigned to separate them. At each step, the controller emits a JSON action. Search actions query a deterministic BM25 retriever over a small per-question candidate corpus, not a global web or production-scale collection. For HotpotQA [27], 2WikiMultiHopQA [9], and MuSiQue [25] examples, the corpus is built from original context paragraphs containing gold supporting paragraphs plus distractors. For FRAMES [14] examples, where such context paragraphs are not available, it uses the trace evidence pool. In the 149-trajectory test corpus, each question has 5 to 20 candidate documents, with mean 10.0 and median 10. Gold-supporting documents are present for mapped source-dataset examples, with mean 2.4 gold documents per question. The controller does not search outside this per-question corpus.

Extraction actions record evidence facts by matching requested facts against retrieved evidence. A fact is counted as evidence-matched if it has a case-insensitive substring match in retrieved evidence, or a token-overlap match of at least 0.6. This is an automatic evidence-linking heuristic, not an independent human or entailment-based factuality audit. After the controller finishes or reaches the step budget, the final-answer generator receives the accumulated state and retrieved evidence through a separated answer prompt.

The maximum controller budget is 12 steps. If the controller does not finish within this budget, the system still calls the final-answer generator from the current accumulated state. Thus, the reported answer quality reflects the separated final-answer prompt rather than a learned early-stopping policy.

3 Experiments

We organize the experiments around three questions. RQ1 asks whether trajectory fine-tuning improves action prediction for the evaluated SLMs and xSLMs. RQ2 asks how the result changes with training-set size and with one multiclass controller versus one binary controller per action. RQ3 asks whether action-prediction

Table 1: Action-prediction results on 1,646 held-out examples. Unp. refers to unparseable

Model	Method	Acc.	Macro-F1	Unp.
Granite 4.1 3B	LoRA SFT	0.7581	0.6536	1
Ministral 3B	LoRA SFT	0.7497	0.6443	0
Qwen 3.5 2B	LoRA SFT	0.7448	0.6414	0
Granite 4.0 1B	LoRA SFT	0.7418	0.6391	1
Qwen 3.5 0.8B	LoRA SFT	0.7179	0.6060	1
Granite 4.0 350M	LoRA SFT	0.6586	0.5683	0
TF-IDF logreg	Baseline	0.7102	0.5399	0
Ministral 3B	LoRA seqcls	0.7861	0.6330	0
Granite 4.1 3B	Zero-shot	0.4399	0.1736	0

gains are reflected in the constrained end-to-end search pipeline. Model and configuration choices were selected using the validation split; all numbers reported below are test-set results unless explicitly described as validation results.

3.1 RQ1: Action Prediction Across Models

Table 1 reports representative 7-way action-prediction results on the same 1,646-example test split. For Granite 4.1 3B, LoRA SFT improves macro-F1 over zero-shot prompting of the same model. Granite 4.1 3B trained on the full 13,194-sample training split reaches test macro-F1 0.6536. Ministral 3B with LoRA sequence classification gives the highest accuracy among the reported results, but lower macro-F1 than the best LoRA SFT results.

The model comparison is descriptive. Granite 4.0 350M reaches macro-F1 0.5683, Granite 4.0 1B reaches 0.6391, and Granite 4.1 3B reaches 0.6536 when trained on the full action split. We use Granite 4.1 for the 3B model because Granite 4.1 does not have 1B or 350M variants. Because the 3B result is therefore also from a newer Granite generation, this comparison should not be interpreted as a controlled parameter-scaling result. The Qwen comparison shows a similar direction, with Qwen 3.5 0.8B at macro-F1 0.6060 and Qwen 3.5 2B at 0.6414 under LoRA SFT.

The learning format also matters. In the 3B-class comparison, LoRA sequence classification gives higher accuracy for Granite and Ministral, while LoRA SFT gives higher macro-F1. We do not treat this as a general method ranking. The `verify` class remains difficult in all settings, with only 17 held-out examples.

3.2 RQ2: Data Size and Action Formulation

Figure 1 shows the training-size sweep for Granite 4.1 3B with LoRA SFT. In the reported sweep, the 3,000-training-sample setting obtains macro-F1 0.6489, close to the best observed value of 0.6625 at 12,000 training samples and to 0.6536 with the full 13,194-sample training split. This suggests that, for this action protocol and accepted-trace distribution, a few thousand supervised trajectory steps can already provide much of the observed action-prediction gain for this SLM. Accuracy and weighted-F1 continue to increase slightly at larger sizes, while macro-F1 peaks at 12,000 training samples and is similar on the full training set. For consistency with the final full-data model, the main Granite LoRA SFT result is reported as macro-F1 0.6536 when trained on all 13,194 training actions. Because the split is class-imbalanced, small changes in rare classes can move macro-F1 without changing overall accuracy much.

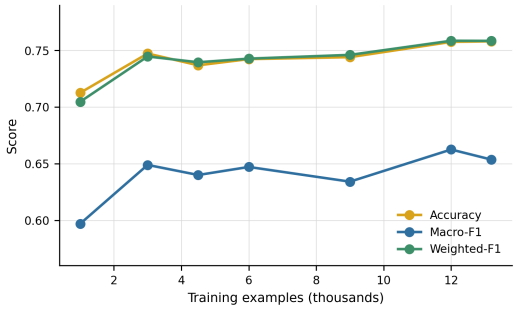


Figure 1: Granite 4.1 3B LoRA-SFT scaling as the number of training samples increases.

Table 2: Multiclass versus binary action formulations with Qwen 3.5 0.8B LoRA SFT.

Formulation	Acc.	Macro-P	Macro-R	Macro-F1
Single multiclass controller	0.7005	0.6196	0.5979	0.6021
Seven binary controllers	n/a	0.6722	0.5561	0.5680
Naive binary ensemble	0.5365	0.4277	0.4048	0.3978

We next evaluate the second RQ2 formulation choice. The default setup treats action prediction as a single seven-class decision over the seven tools; we compare this multiclass controller against one binary LoRA SFT controller per action using Qwen 3.5 0.8B. Table 2 summarizes the aggregate results. Accuracy is not reported for the independent binary controllers because they are evaluated one-vs-rest and do not by themselves produce a single action label per sample. The single multiclass controller gives higher macro-F1 and higher macro-recall. The independent binary controllers give higher macro-precision, but they lose recall on several low-support or confusable classes. A naive ensemble that selects the action with the highest binary "yes" score performs worse than the multiclass controller.

Direct classification performed worse than SFT: The binary formulation has higher macro-precision but lower macro-recall and macro-F1. The weak naive ensemble indicates that direct comparison of independently trained binary scores is ineffective in this setting. For the end-to-end pipeline, we therefore use a single multiclass LoRA SFT controller.

3.3 RQ3: End-to-End Search

We evaluate the selected Granite 4.1 3B models in an end-to-end controller/generator swap pipeline. The controller builds the search state through structured actions, and the final-answer generator is then called with the same separated answer prompt over the accumulated state and retrieved evidence. This design evaluates the base and fine-tuned models in four role combinations: base/base, fine-tuned/base, base/fine-tuned, and fine-tuned/fine-tuned. The base/base and fine-tuned/fine-tuned conditions correspond to using one model for both roles, while the cross-role conditions isolate the controller and generator roles. Each condition was run five times with different random seeds on the same 149 held-out accepted test

Table 3: End-to-end controller/generator swap evaluation.

Controller	Generator	EM	F1	Ev.-fact rate
Base	Base	0.7530	0.7783	0.0121
Fine-tuned	Base	0.7664	0.8044	0.6161
Base	Fine-tuned	0.7691	0.7997	0.0134
Fine-tuned	Fine-tuned	0.7946	0.8295	0.6389

trajectories, using stochastic decoding with temperature 0.7, the same BM25 retrieval environment, and `MAX_STEPS=12`.

We evaluate the final answer with SQuAD-style Exact Match and token F1 against the gold answer and aliases. These metrics measure whether the completed pipeline produces the expected answer string. To describe the controller's intermediate behaviour, we also report evidence-fact rate: the fraction of trajectories in which at least one extracted intermediate fact is automatically matched to retrieved evidence. This diagnostic measures whether the controller records evidence-grounded intermediate facts before final answer generation. It is based on the extraction heuristic described in Section 2, and should not be interpreted as a human or entailment-based factuality judgment.

Table 3 shows that using the fine-tuned model for both roles improves answer quality relative to base/base in this separated-prompt pipeline: EM increases from 0.7530 to 0.7946, and token F1 increases from 0.7783 to 0.8295. The paired bootstrap interval for the combined token-F1 gain is [0.0021, 0.1040], while the Exact Match interval crosses zero. The cross-role conditions show that the fine-tuned controller has the clearest effect on evidence recording. With the base generator fixed, replacing the base controller with the fine-tuned controller increases the evidence-fact rate by 0.60 and the mean number of evidence facts by 2.25. With the fine-tuned generator fixed, the same controller replacement increases the evidence-fact rate by 0.62 and the mean number of evidence facts by 2.28. The corresponding controller-only answer-quality deltas are smaller and have bootstrap intervals crossing zero. Thus, the end-to-end result supports a controller effect on evidence-fact recording and a modest combined improvement when both roles use the fine-tuned model, but it does not establish a statistically clear controller-only improvement in final-answer quality.

The 149 test trajectories contain 105 2WikiMultiHopQA, 41 HotpotQA, 2 FRAMES, and 1 MuSiQue examples. FRAMES and MuSiQue have too few examples for dataset-level conclusions.

4 Discussion

The experiments support three main observations. First, action prediction is learnable for this seven-action protocol. Granite 4.1 3B trained on the full 13,194-sample training split improves substantially over zero-shot prompting of the same backbone, and other evaluated LoRA-SFT models obtain macro-F1 between 0.5683 and 0.6443. Second, the task is affected by class imbalance. Macro-F1, accuracy, and weighted-F1 do not always select the same method, and the rare `verify` class remains unreliable. Third, the end-to-end controller/generator swap shows that the trained controller increases evidence-matched intermediate facts under the automatic heuristic when the generator is held fixed. The answer-quality changes from replacing only the controller are modest and have bootstrap intervals crossing zero, while using the fine-tuned model for both roles gives a small token-F1 gain over base/base.

We also evaluate the base and fine-tuned Granite 4.1 3B models on three external multiple-choice benchmarks as a general-capability retention check. Under the same `lm-evaluation-harness` protocol [5], fine-tuning reduces MMLU accuracy [8] from 0.6554 to 0.5520, HellaSwag [30] from 0.7911 to 0.6852, and ARC-Challenge [7] from 0.6502 to 0.5589, indicating a measurable trade-off on these non-trajectory tasks. Because training is adapter-based, the controller adapter can be used for search-flow control while the base model remains available for generation; in the end-to-end swap, however, using the fine-tuned model for both controller and generator still gives the strongest answer scores among the evaluated role combinations.

The results should be interpreted with several constraints. The evaluation set is acceptance-filtered: both action prediction and end-to-end QA are measured on questions for which the teacher produced an accepted trajectory. Action prediction measures agreement with teacher-labeled targets, not proof that the predicted action is uniquely correct. The retrieval environment is a per-question candidate corpus with gold supporting paragraphs and distractors where available, not an open retrieval collection. The end-to-end evaluation uses a separated final-answer prompt, so its answer scores characterize this specific pipeline rather than all possible agent finalization interfaces. The evidence-fact metric is based on an automatic matching heuristic and should not be read as human-verified factuality.

5 Conclusion

We evaluated trajectory fine-tuning for search-control actions in SLMs and xSLMs. On an acceptance-filtered held-out 7-way action-prediction task, Granite 4.1 3B LoRA SFT trained on the full 13,194-sample training split reaches macro-F1 0.6536, improving over zero-shot prompting and a TF-IDF baseline. In the reported training-size sweep, the 3,000-training-sample setting obtains macro-F1 0.6489, close to the best observed value of 0.6625, and a single multiclass controller is stronger in aggregate than independent binary action controllers for the evaluated Qwen 3.5 0.8B setting. In the end-to-end controller/generator swap, using the fine-tuned Granite 4.1 3B model for both roles improves token F1 relative to base/base, and replacing only the controller substantially increases evidence-fact recording under a fixed generator. The evidence supports a focused conclusion: trajectory supervision can improve action prediction and evidence-fact recording behaviour for an SLM in this evaluated pipeline, while unfiltered-query evaluation and robust final-answer generation remain open issues.

6 GenAI Usage Disclosure

The authors used GenAI to edit and improve text written for this paper. All AI-generated text was carefully reviewed prior to submission. In addition, GenAI-assisted code support tools were employed during development. Specific implementation-related questions were also posed to chat-based GenAI models, and the responses were used as a basis for adapting the code accordingly.

References

[1] 2025. *MiniCPM-V 4.5: Cooking Efficient MLLMs via Architecture, Data, and Training Recipe.* https://arxiv.org/abs/2509.18154

[2] Liquid AI. 2025. LFM2 Technical Report. *arXiv preprint arXiv:2511.23404* (2025).

[3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, Vol. 2024. 9112–9141.

[4] Marcia J. Bates. 1989. The design of browsing and berrypicking techniques for the online search interface. *Online Review* 13, 5 (1989), 407–424.

[5] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, et al. 2024. Lessons from the Trenches on Reproducible Evaluation of Language Models. arXiv:2405.14782 [cs.CL] https://arxiv.org/abs/2405.14782

[6] Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. 2024. Rq-rag: Learning to refine queries for retrieval augmented generation. *arXiv preprint arXiv:2404.00610* (2024).

[7] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457 [cs.AI] https://arxiv.org/abs/1803.05457

[8] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*. https://openreview.net/forum?id=d7KBjmI3GmQ

[9] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Barcelona, Spain (Online), 6609–6625. doi:10.18653/v1/2020.coling-main.580

[10] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[11] IBM Research. 2025. Granite 4.0 Language Models. https://github.com/ibm-granite/granite-4.0-language-models. Accessed: 2025-10-01.

[12] IBM Research. 2026. Granite 4.1 Language Models. https://huggingface.co/blog/ibm-granite/granite-4-1. Accessed: 2026-04-28.

[13] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516* (2025).

[14] Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqui. 2024. Fact, Fetch, and Reason: A Unified Evaluation of Retrieval-Augmented Generation. arXiv:2409.12941 [cs.CL] https://arxiv.org/abs/2409.12941

[15] Carol C. Kuhlthau. 1991. Inside the search process: Information seeking from the user's perspective. *Journal of the American Society for Information Science* 42, 5 (1991), 361–371.

[16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[17] Alexander H Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, Adrien Sadé, Alan Jeffares, Albert Jiang, Alexandre Cahill, Alexandre Gavaudan, et al. 2026. Ministral 3. *arXiv preprint arXiv:2601.08584* (2026).

[18] Gary Marchionini. 2006. Exploratory search: From finding to understanding. *Commun. ACM* 49, 4 (2006), 41–46.

[19] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. Augmented Language Models: a Survey. arXiv:2302.07842 [cs.CL] https://arxiv.org/abs/2302.07842

[20] Team MiniCPM. 2025. Minicpm4: Ultra-efficient llms on end devices. *arXiv preprint arXiv:2506.07900* (2025).

[21] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5

[22] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250* (2016).

[23] Stephen E. Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Found. Trends Inf. Retr.* 3, 4 (2009), 333–389. doi:10.1561/1500000019

[24] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[25] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554. doi:10.1162/tacl_a_00475

[26] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 10014–10037.

[27] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, 2369–2380. doi:10.18653/V1/D18-1259

[28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

[29] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, et al. 2024. MiniCPM-V: A GPT-4V Level MLLM on Your Phone. *arXiv preprint arXiv:2408.01800* (2024).

[30] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 4791–4800. doi:10.18653/v1/P19-1472